



# Linux e la shell Bash

---

## Gestione file e directory

## Sistemi Operativi

## Laurea Triennale in Ingegneria Informatica e dell'Automazione, A.A. 2024/2025

# Indice

---

- Il filesystem di Linux
- Permessi di accesso
  - chmod, chown, chgrp
- Comandi per la gestione di file e directory
  - pwd, cd, touch, ls, mkdir, rmdir, cp, mv, rm, file, stat
- Hard e soft link
  - ln
- Esercizi
  - Soluzioni

# Il filesystem di Linux (1/3)

- In Linux, **tutto è un file**.

- Una **directory** è un file contenente un **directory entry** per ciascun file in essa contenuto.
- Una **periferica di memorizzazione** è un file speciale a blocchi che supporta lettura e scrittura bufferizzate.
- Un **terminale** è un file speciale a caratteri.

- Questa metafora consente di:

- Creare **collegamenti** tra file, directory, dispositivi, ecc.
- Attribuire **permessi** a file, directory, dispositivi, ecc.
- **Leggere e scrivere** da/su file, directory e dispositivi.

# Il filesystem di Linux (2/3)

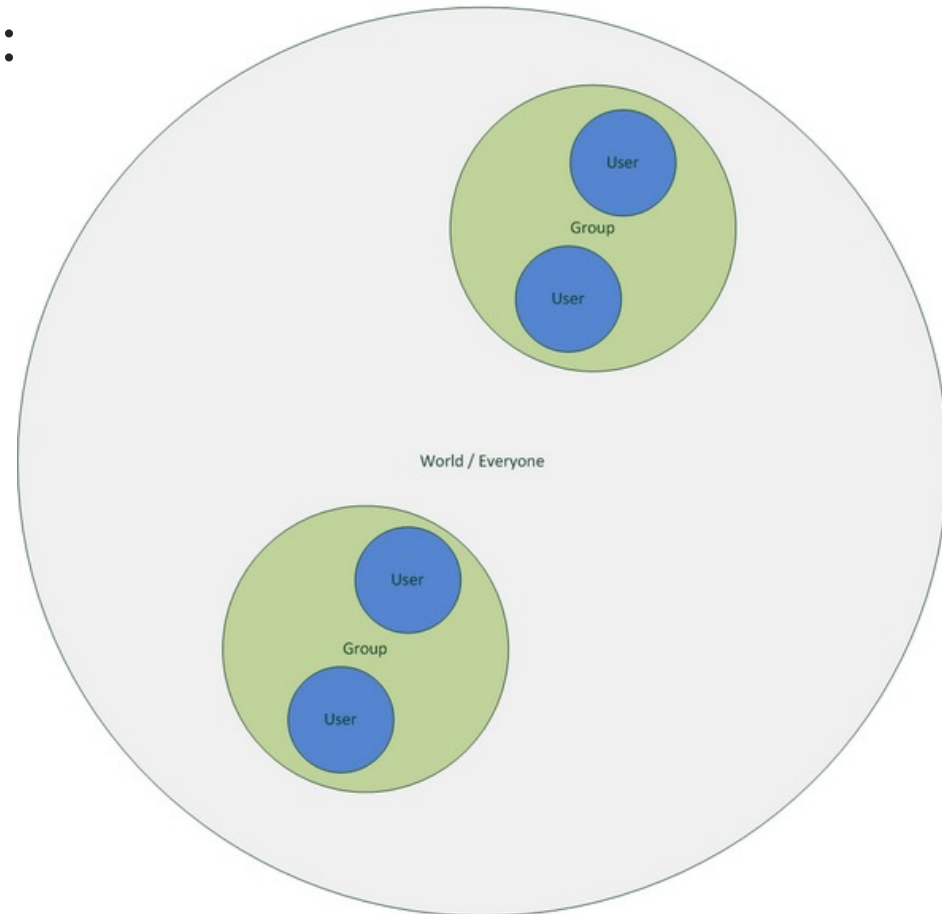
- Linux segue il [Filesystem Hierarchy Standard](#).
  - **/:** directory radice (root).
  - **/bin:** programmi utilizzabili da tutti gli utenti.
  - **/sbin:** programmi per l'amministrazione di sistema.
  - **/boot:** file necessari all'avvio del sistema (bootloader, kernel, ...).
  - **/dev:** file speciali associati a dispositivi hardware.
  - **/etc:** file di configurazione.
  - **/home:** home directory di ciascun utente.
  - **/root:** home directory dell'utente amministratore (root).
  - **/tmp:** file temporanei.
  - **/usr:** programmi e librerie utente, file di sola lettura.
  - **/var:** file che sistema e programmi scrivono a runtime.
  - **/lib:** librerie di sistema e moduli del kernel.
  - **/opt:** pacchetti software opzionali.
  - **/proc:** informazioni sui processi e sul sistema.
  - **/media:** dispositivi removibili montati dal sistema (CD, pen drive, ...).
  - **/mnt:** dispositivi temporanei montati dall'utente.

# Il filesystem di Linux (3/3)

- I nomi di file e directory possono contenere qualunque sequenza di caratteri, eccetto `/`.
- La struttura ad albero consente di identificare univocamente un elemento del filesystem mediante il suo **percorso assoluto** a partire dalla directory radice.
- Le directory del percorso sono separate dal carattere `/`.
- Ogni directory contiene due **directory speciali** (hard link):
  - `"."`, che punta alla directory stessa.
  - `".."`, che punta alla directory padre.
- La directory padre di `/` è `/` stessa.
- Oltre a supportare i percorsi assoluti, la shell consente di utilizzare **percorsi relativi**, tipicamente riferiti alla directory corrente. Un percorso relativo non inizia con `/`.

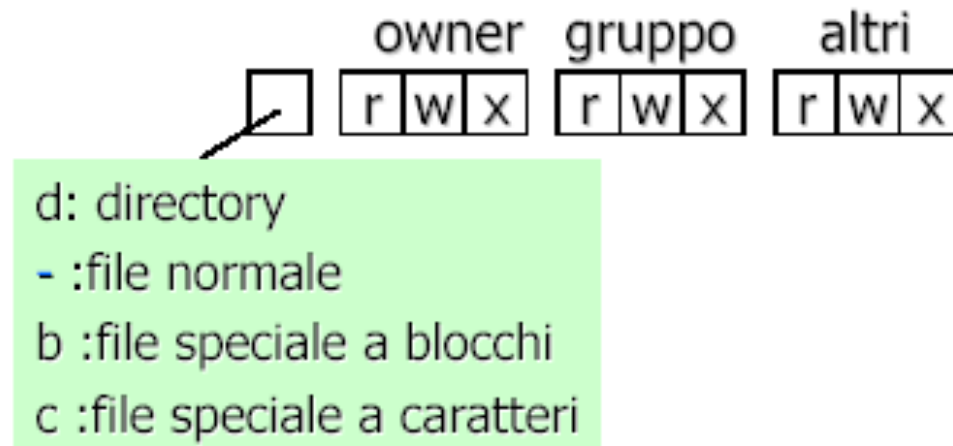
# Permessi di accesso

- Ad un file possono essere attribuiti i seguenti permessi:
  - Lettura (**r**ead)
  - Scrittura (**w**rite)
  - Esecuzione (**e**xecute)
- Quando un file viene creato, esso viene collegato all'utente che lo ha creato e al gruppo principale di cui fa parte.
- I permessi sono definiti per:
  - **Utente proprietario** del file.
  - **Gruppo** a cui appartiene il file.
  - **Other**, tutti gli altri utenti.



# Access Control List

- Ad ogni file/directory è associata una stringa di **10** caratteri, detta lista di controllo agli accessi o **Access Control List (ACL)**, che codifica i permessi di accesso.
- I caratteri di una ACL sono suddivisi in quattro gruppi:
  - **l**: tipo di file.
  - **u**: permessi concessi al proprietario del file.
  - **g**: permessi concessi agli utenti membri del gruppo.
  - **o**: permessi agli altri utenti, non compresi nei precedenti insiemi.



# Tipi di file

---

- **-**: file ordinario.
- **d**: directory.
- **b**: file speciale a blocchi.
  - Forniscono un'interfaccia bufferizzata con blocchi di lunghezza fissa per l'I/O da dispositivi di memorizzazione (hard disk, CD/DVD, ecc.).
- **c**: file speciale a caratteri.
  - Forniscono un'interfaccia non bufferizzata per l'I/O verso dispositivi seriali come schede audio, terminali, porte seriali, ecc.
- **l**: collegamento simbolico.
  - Rappresenta un collegamento ad un file presente altrove.



# Significato dei permessi

## ○ File ordinari:

- **r**: leggere il contenuto del file.
- **w**: modificare il contenuto del file.
- **x**: eseguire il file (ha senso per programmi e script).

## ○ Directory:

- **r**: leggere il contenuto della directory (elenco di file in essa contenuti).
- **w**: modificare la directory (aggiungere, rimuovere, rinominare file).
- **x**: accesso (scansione) alla directory (entrare nella directory e leggere, modificare, eseguire file in essa contenuti).

## ○ File speciali:

- **r**: leggere dal device (input).
- **w**: scrivere sul device (output).
- **x**: non significativo.

- **Nota:** in una ACL, l'assenza di un permesso è identificata da un trattino (-).

# Esempi

- **- rwx rw- r--**
  - File ordinario (- iniziale).
  - L'utente proprietario può leggere, scrivere ed eseguire il file.
  - Il gruppo proprietario può leggere e scrivere, ma non eseguire il file.
  - Gli altri utenti possono solo leggere il file.
- **d rwx r-x r-x**
  - Directory (d iniziale).
  - Il proprietario può leggere, modificare ed accedere alla directory.
  - Tutti gli altri utenti (inclusi quelli del gruppo proprietario) possono solo leggerne il contenuto ed accedervi.
- Al momento della creazione, i permessi di **default** sono:
  - **- rw- r-- r--**: i file ordinari possono essere letti e modificati dal proprietario, e solo letti da tutti gli altri.
  - **d rwx r-x r-x**: le directory possono essere lette, modificate e scansionate dal proprietario, e solo lette e scansionate dagli altri.

# chmod

- **Change mode.**
- Permette la **modifica dei permessi di accesso** di file.
  - Può essere eseguito solo dal **proprietario** o dall'utente **root**.
- **Sintassi:**
  - `chmod [-R] <mode> <file> [file] ...`
  - *mode* può essere in **formato simbolico** o **ottale**.
- **Flag:**
  - **-R**: esegue l'operazione ricorsivamente, per tutte le eventuali sottodirectory e file.

# Permessi - forma simbolica

- Combinazione di **codici utente**, **codici permesso** e **operatori**.
- **Codici utente:**
  - **u, g, o**: utente proprietario, gruppo proprietario, altri.
  - **a**: tutti, equivalente a **ugo**.
- **Codici di permesso:**
  - **r, w, x**: lettura, scrittura, esecuzione.
- **Operatori:**
  - **+, -, =**: aggiunge, rimuove, o assegna esattamente un permesso.
- **Esempi:**
  - **chmod g+w file.txt**: fornisce il privilegio di scrittura agli utenti appartenenti al gruppo proprietario.
  - **chmod ugo+rw** (o **a+rw**, o ancora **+rw**) **file.txt**: fornisce i privilegi di lettura e scrittura a qualsiasi utente.
  - **chmod o-x file**: rimuove i permessi di esecuzione per chi non è né proprietario, né fa parte del gruppo proprietario.

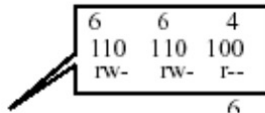
# Permessi - forma ottale

- Costituita da **tre cifre ottali** (comprese tra 0 e 7), una per ciascun permesso ammissibile.
- Il formato binario di ciascuna cifra ottale codifica tre bit dell'ACL.
  - I permessi variano da 000 a 777.

| <b>r</b> | <b>w</b> | <b>x</b> | <b>Cifra in forma ottale</b> | <b>significato</b>           |
|----------|----------|----------|------------------------------|------------------------------|
| 0        | 0        | 0        | 0                            | Nessun permesso              |
| 0        | 0        | 1        | 1 ( $2^0$ )                  | Esecuzione solo              |
| 0        | 1        | 0        | 2 ( $2^1$ )                  | Scrittura solo               |
| 0        | 1        | 1        | 3 ( $2^1+2^0$ )              | Scrittura ed esecuzione      |
| 1        | 0        | 0        | 4 ( $2^2$ )                  | Lettura solo                 |
| 1        | 0        | 1        | 5 ( $2^2+2^0$ )              | Lettura ed esecuzione        |
| 1        | 1        | 0        | 6 ( $2^2+2^1$ )              | Lettura e scrittura          |
| 1        | 1        | 1        | 7 ( $2^2+2^1+2^0$ )          | Lettura scrittura esecuzione |

# Esempi

## Permessi in forma ottale:



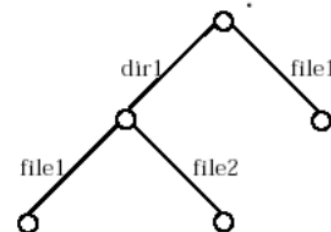
```
% chmod 664 file1 file2
% ls -l
total 4
-rw-rw-r-- 1 roberto usrmail 35 Mar 11 16:34 file1
-rw-rw-r-- 1 roberto usrmail 17 Mar 11 16:17 file2
```

## Permessi in forma simbolica:

```
% ls -l
total 4
-rw-rw-r-- 1 roberto usrmail 35 Mar 11 16:34 file1
-rw-rw-r-- 1 roberto usrmail 17 Mar 11 16:17 file2
% chmod ugo+x file1
% chmod o=rwx file2
% ls -l
total 4p
-rwxrwxr-x 1 roberto usrmail 17 Mar 11 16:34 file1
-rw-rw-rwx 1 roberto usrmail 17 Mar 11 16:17 file2
```

```
% ls -R
dir1 file1

./dir1:
file1 file2
% chmod u-rwx dir1 file1
% rm file1
rm: file1: override protection 44 (y/n)? n
% rm -r dir1
rm: cannot read directory dir1: Permission
denied
% ls dir1
can not access directory dir1
% cd dir1
dir1: Permission denied
% cp file1 dir1
cp: cannot open file1: Permission denied
```



# chown

- **Change owner.**
- Modifica l'**utente** e/o **gruppo proprietario** di uno o più file.
  - Utilizzabile solo dall'utente **root**.
- **Sintassi:**
  - `chown [-R] [utente] [.] [gruppo] <file> [file] ...`
- **Flag:**
  - **-R:** esegue l'operazione ricorsivamente, per tutte le eventuali subdirectory e file in esse contenuti.
- **Esempi:**
  - `[sudo] chown utente1 file.txt`: modifica l'utente proprietario del file in utente1.
  - `[sudo] chown -R utente1.gruppo1 directory`: modifica utente e gruppo proprietario per la directory e tutti i suoi file e subdirectory.
  - `[sudo] chown .gruppo1 file.txt`: modifica il gruppo proprietario.

# chgrp

- **Change group.**
- Modifica il **gruppo proprietario** di uno o più file.
  - Utilizzabile solo dall'utente **root**.
- **Sintassi:**
  - `chgrp [-R] <gruppo> <file> [file] ...`
- **Flag:**
  - **-R**: esegue l'operazione ricorsivamente, per tutte le eventuali subdirectory e file in esse contenuti.
- **Esempi:**
  - `[sudo] chgrp gruppo1 file.txt`: modifica il gruppo proprietario di file.txt in gruppo1.



# pwd

---

- Print **w**orking **d**irectory.
- Stampa su standard output il percorso assoluto della **directory di lavoro** corrente.
- **Sintassi:**
  - `pwd`
- **Esempi:**
  - `user@host:~$ pwd`  
`/home/user`

# cd

- Change **d**irectory.
- Modifica la **directory di lavoro** corrente.
- **Sintassi:**
  - `cd [percorso]`
- **Esempi:**
  - `cd`: modifica la directory di lavoro con la home directory dell'utente attualmente loggato. Equivalente a `cd ~`.
  - `cd -`: modifica la directory di lavoro con quella precedente.
  - `cd .`: nessun effetto, resta nella directory corrente.
  - `cd ..`: modifica la directory di lavoro con il padre di quella corrente.

# ls

- **List.** Elenca il contenuto di una directory.
- **Sintassi:**
  - `ls [flag] [percorso]`
- **Flag:**
  - **-a:** mostra tutti i file, inclusi quelli nascosti.
  - **-F:** accoda un identificativo di tipo a ciascun file (\* file eseguibile, / directory, @ link simbolico, ecc.).
  - **-r:** inverte l'ordine di visualizzazione delle righe.
  - **-R:** visualizza il contenuto delle subdirectory ricorsivamente.
  - **-t:** ordina i file per data di ultima modifica, dal più recente.
  - **-u:** ordina i file per data di ultimo accesso, dal più recente.
    - Se combinato con **-l** va aggiunto il flag **-t** per ordinare.
  - **-i:** mostra l'**inode number** del file.
  - **-1:** visualizza l'elenco in una singola colonna.
  - **-l:** mostra informazioni dettagliate nel seguente formato:  
*[ACL] [# hard link] [utente] [gruppo] [dimensione (b)] [ultima modifica] [nome]*

# touch

- Aggiorna il timestamp (data ed ora) di **accesso** e **modifica** ad un file a data ed ora correnti.
- Se il file non esiste, ne viene creato uno vuoto.
- **Sintassi:**
  - `touch [flag] <percorso> ...`
- **Flag:**
  - **-a:** aggiorna solo il timestamp di accesso.
  - **-m:** aggiorna solo il timestamp di modifica.
  - **-c:** non crea alcun file.
- **Esempi:**
  - **touch myfile:** se *myfile* esiste, ne aggiorna timestamp di accesso e modifica, altrimenti crea *myfile*.
  - **touch -ca myfile:** aggiorna solo il timestamp di accesso di *myfile*. Se *myfile* non esiste, non viene creato.

# mkdir

- **Make directory.**
- Crea una **nuova directory**.
- **Sintassi:**
  - `mkdir [flag] <percorso> ...`
- **Flag:**
  - `-m <ACL>`: specifica l'access control list desiderato.
  - `-p`: nessun messaggio di errore se la directory esiste, e crea le directory padre se mancanti.
  - `-v`: modalità verbosa, stampa messaggi anche in caso di successo.
- **Esempi:**
  - `mkdir mydir`: crea la directory *mydir*.
  - `mkdir mydir1 mydir2`: crea le directory *mydir1* e *mydir2*.
  - `mkdir -m 766 mydir`: crea *mydir* con i permessi specificati.
  - `mkdir -p mydir1/mydir2`: crea *mydir2* all'interno di *mydir1*; se *mydir1* non esiste, viene create automaticamente.

# rmdir

---

- Remove **dir**ectory.
- **Cancella una directory**, solo nel caso in cui sia **vuota**.
- **Sintassi:**
  - `rmdir [flag] <percorso> ...`
- **Flag:**
  - **--ignore-fail-on-non-empty**: sopprime messaggi di errore relativi al fatto che la directory non sia vuota.
  - **-v**: modalità verbosa, stampa messaggi anche in caso di successo.

- **Copy.**
- **Copia file e directory.**
  - Se il percorso di destinazione è una directory, copia al suo interno mantenendo il nome originario del file; diversamente, il file viene copiato con un nuovo nome.
- **Sintassi:**
  - `cp [flag] <percorso_origine> ... <percorso_destinazione>`
- **Flag:**
  - **-n:** non sovrascrivere file esistenti (di default sono sovrascritti).
  - **-i:** chiede interattivamente conferma prima di sovrascrivere.
  - **-f:** se un file esistente non può essere aperto in scrittura, viene rimosso e l'operazione di copia viene ripetuta.
  - **-r:** copia una directory ricorsivamente.
  - **-v:** attiva la modalità verbosa.

# mv

- **Move.**
- **Sposta file e directory.**
  - Rinominare un file è un caso speciale di spostamento.
  - Di default sovrascrive file esistenti.
- **Sintassi:**
  - `mv [flag] <percorso_origine> ... <percorso_destinazione>`
- **Flag:**
  - `-i`: chiede conferma in caso di sovrascrittura.
  - `-f`: forza la sovrascrittura anche se non si possiedono i permessi di scrittura sul file destinazione (di default chiede conferma).
  - `-v`: attiva la modalità verbosa.



- **Remove.**
- **Rimuove file e directory.**
  - Di default, le directory vengono rimosse **solo se vuote**. Se contengono file è necessario specificare il flag **-r**.
  - Se non si possiedono privilegi di scrittura, richiede interattivamente di sovrascrivere i permessi, a meno che non venga specificato **-f**.
- **Sintassi:**
  - `rm [flag] <percorso> ...`
- **Flag:**
  - **-r**: cancella directory ricorsivamente, anche se non vuote.
  - **-i**: chiede conferma per ciascun file rimosso.
  - **-f**: ignora file non esistenti e non chiedere mai conferma.
  - **-v**: attiva la modalità verbosa.

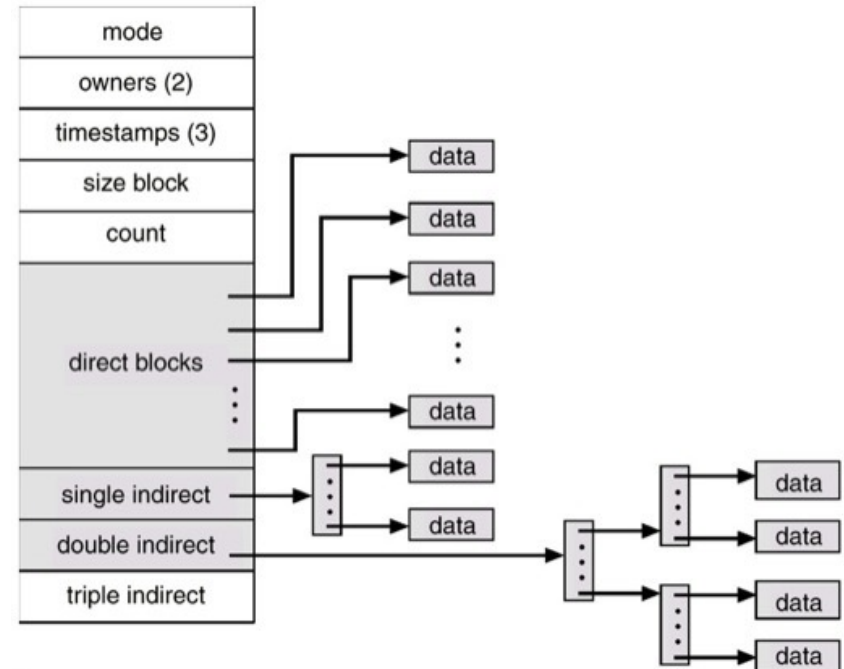
# file, stat

- **file**: determina il tipo di un file.
- **Sintassi**:
  - `file [flag] <percorso> ...`
- **stat**: mostra informazioni dettagliate su un file.
- **Sintassi**:
  - `stat [flag] <percorso> ...`

```
ivano@debian-ivano:~/so$ file star_wars.txt
star_wars.txt: ASCII text
ivano@debian-ivano:~/so$ stat star_wars.txt
  File: star_wars.txt
  Size: 276          Blocks: 8          IO Block: 4096   regular file
Device: 801h/2049d  Inode: 918269       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   ivano)   Gid: ( 1000/   ivano)
Access: 2020-12-08 16:07:54.647659451 +0100
Modify: 2019-10-28 18:26:32.113315954 +0100
Change: 2019-10-29 15:43:36.778988058 +0100
 Birth: -
```

# Link (1/4)

- Tutti gli elementi (file, directory, link, ecc.) di un file system UNIX-like sono descritti da una struttura dati chiamata **inode**.
- Ogni inode contiene **attributi** (tempi di accesso, modifica, proprietario, permessi, link count, ecc.) e puntatori ai **data block**, dove è memorizzato il contenuto di ciascun file.
- Esiste una corrispondenza biunivoca tra file ed inode, ed a ciascun inode è associato un identificativo (**inode number**).
- I data block delle directory sono anche detti **directory block**, e contengono un **directory entry** (*inode number, filename*) per ciascun elemento in esse contenuto.



# Link (2/4)

---

- Un riferimento ad un file è detto **link**. UNIX prevede due tipologie di link: **hard link** e **soft link**.
- Un **hard link** è un puntatore all'inode di un file esistente. Creare un hard link corrisponde ad aggiungere un nuovo directory entry (*inode number, filename*) per un determinato inode.
- Un **soft link** (*symbolic link, symlink*) è un file speciale i cui data block contengono il percorso assoluto di un altro file. Quando viene incontrato un symlink, il sistema lo risolve saltando al percorso del file originale.

# Link (3/4)

- Le directory hanno un numero di hard link almeno pari a 2, corrispondenti al directory entry posseduto dalla **directory padre**, e dal link alla **directory stessa** ("."). Ulteriori hard link impliciti sono creati ogni volta che si crea una sotto-directory ("..").
- Creare o rimuovere soft link non ha effetti sull'inode del file originale. Viceversa, gli hard link ne influenzano il **link count**:
  - Ogni nuovo hard link **incrementa** il link count di 1.
  - Rimuovere un hard link (con **rm**) lo **decrementa** di 1.
  - Se il link count diventa pari a 0, l'inode viene **deallocato**, e con esso vengono liberati i data block referenziati (il file viene "cancellato").

# Link (4/4)

- **Un hard link è indistinguibile dal file di partenza**, poiché corrisponde essenzialmente ad un nuovo nome per il medesimo inode. **Un soft link è invece un file distinto**, con proprio inode.
- **Non è possibile creare hard link a file presenti su filesystem diversi** (es: diverse partizioni o dischi) poiché condurrebbe ad ambiguità (su che filesystem si trova l'inode referenziato?). Questa limitazione non esiste per i soft link.
- La maggior parte delle implementazioni UNIX moderne **non consente di creare hard link espliciti a directory**, onde evitare la ricorsione infinita che si avrebbe in caso di link che puntano a directory antenate (es: la directory padre). Questa limitazione non esiste per i soft link.

- **Link.**
- Crea collegamenti tra file.
  - Di default crea hard link.
- **Sintassi:**
  - `ln [flags] <percorso_origine> ... <percorso_destinazione>`
- **Flag:**
  - `-s`: crea un soft link invece che un hard link.
- **Esempi:**
  - `ln myfile1 myfile2`: Crea un hard link di nome myfile2 a myfile1.
  - `ln -s myfile1 myfile2`: Crea un soft link di nome myfile2 a myfile1.
  - `ln myfile1 myfile2 mydir`: Crea hard link a myfile1 e myfile2 in mydir.

# Esercizi (1/3)

1. Spostarsi nella directory `"/home/user/dir1/dir11"` supponendo di trovarsi in `"/home/user/dir2/dir21"`, utilizzando i percorsi assoluto e relativo.
2. Spiegare il significato dell'ACL `"lrwxr-xr-x"` relativa ad un file di nome ***"myfile"***.
  1. Convertire la componente di permessi dell'ACL in formato ottale.
  2. Scrivere il comando atto a rimuovere i privilegi di esecuzione per gli utenti diversi dal proprietario e non facenti parte del gruppo del file.
  3. Specificare quali siano i nuovi permessi in formato ottale.
3. Verificare in quale directory si trovi l'utente.  
Successivamente, rinominare il file ***"myfile1"*** presente nella directory corrente in ***"myfile2"***, spostandolo nella directory padre della directory corrente.



# Esercizi (2/3)

4. Usando la notazione letterale, si modifichino i permessi del file "**myfile**" in modo che risultino essere equivalenti a **564**.
5. Il comando `ls -F` produce il seguente output:  
*file1 file2 file3 file4/*  
Indicare quale dei seguenti comandi è esatto, motivando la risposta: **1)** `mv file1 file2 file3`    **2)** `mv file1 file2 file4`
6. Dato il file "**myfile**" dotato dei permessi di accesso "**-r-xr--rwx**", indicare come cambiano tali permessi quando vengono lanciati i seguenti comandi (**Nota:** specificarlo per ciascun passaggio):
  1. `chmod 755 myfile`
  2. `chmod g-r+w myfile`
  3. `chmod o-x myfile`

# Esercizi (3/3)

## 7. Eseguire le seguenti operazioni:

1. Spostarsi nella home directory e creare le directory "**uno**" e "**due**".
2. Copiare il file **"/etc/profile"** nella directory "**uno**".
3. Copiare il file **"/etc/profile"** nella directory "**due**", cambiandone il nome in "**profile\_copia**".
4. Spostare il file "**profile**" nella directory "**due**" ed il file "**profile-copia**" nella directory "**uno**", rinominandolo in "**profile\_bak**".
5. Cancellare i due file con un solo comando.
6. Cancellare le due directory con un solo comando.

## 8. Creare il seguente albero di directory e file all'interno della directory corrente utilizzando al più due comandi.

```
dir1
├── dir2
│   ├── file2.txt
│   └── file3.txt
├── dir3
│   └── file4.txt
└── file1.txt
```

# Soluzioni (1/3)

## 1. Comandi:

- `cd /home/user/dir1/dir11`
- `cd ../../dir1/dir11`

## 2. Il file è un **collegamento simbolico** con i seguenti privilegi: **utente** - lettura, scrittura, esecuzione; **gruppo** - lettura, esecuzione; **altri** - lettura, esecuzione.

1. Maschera ottale: 755
2. Comando: `chmod o-x myfile`
3. Nuova maschera ottale: 644

## 3. Comandi:

- `pwd`
- `mv myfile1 ../myfile2`

## 4. Comando:

- `chmod u=rx,g=rw,o=r myfile`

# Soluzioni (2/3)

5. Il comando corretto è **2**: il primo comando fallirebbe perché **"file3"** esiste già e non è una directory.

6. Soluzione:

- ***r-x r-- rwx (547)***  
u: lettura, esecuzione; g: lettura; o: lettura, scrittura, esecuzione
- ***547 → 755 (rwx r-x r-x)***  
u: lettura, scrittura, eseg.; g: lettura, eseg.; o: lettura, eseg.
- ***755 → g-r+w → rwx -wx r-x (735)***  
u: lettura, scrittura, eseg.; g: scrittura, eseg.; o: lettura, eseg.
- ***735 → o-x → rwx -wx r-- (734)***  
u: lettura, scrittura, esecuzione; g: scrittura, esecuzione; o: lettura

# Soluzioni (3/3)

## 7. Comandi:

- `cd`
- `mkdir uno due`
- `cp /etc/profile uno`
- `cp /etc/profile due/profile_copia`
- `mv uno/profile due`
- `mv due/profile_copia uno/profile_bak`
- `rm uno/profile_bak due/profile`
- `rmdir uno due`

## 8. Comandi:

- `mkdir -p dir1/dir2 dir1/dir3`
- `touch dir1/file1.txt dir1/dir2/file{2,3}.txt dir1/dir3/file4.txt`